

LIKHONA ZONYANE

ST10388321

PDDA0801, GROUP 1

***PROGRAMMING FOR DATA
ANALYTICS 2, PDAN8412w***

PART 1

REPORT

DUE DATE: 07 SEPTEMBER 2026

Contents

1. Introduction and Objective	3
2. Justification of the Suitability of the Dataset	3
3. Analysis Plan and Data Preparation.....	4
4. Exploratory Data Analysis	7
5. Model Training	11
6. Evaluation and Interpretation	13
7. Limitations and Business Recommendation.....	16
8. Conclusion.....	17
REFERENCES.....	18

1. Introduction and Objective

This analysis investigated whether a Long Short-Term Memory (LSTM) neural network could identify the author of a short literary passage. The proposed business application is a “Guess the Author” feature for Large Book Press. The task was treated as a supervised, three-class text-classification problem. The model was required to distinguish among Edgar Allan Poe (EAP), H. P. Lovecraft (HPL) and Mary Shelley (MWS). Apache Spark was used for data loading, cleaning and exploratory data analysis (EDA). TensorFlow/Keras was used to train the LSTM models, while scikit-learn was used for evaluation. The purpose was not only to produce a prediction score, but also to determine whether the model learned a useful representation of authorial style. The report therefore considers data quality, class-level performance and prediction errors in addition to overall accuracy.

2. Justification of the Suitability of the Dataset

The selected dataset contained **19,579 observations and three variables: id, text, and author**. The id variable uniquely identified an observation, text contained the written passage, and author identified the author responsible for the passage.

The dataset therefore exceeded the portfolio requirement of at least 10,000 observations and provided enough examples for training and evaluating a neural-network classification model.

Three authors were represented:

- **EAP** – Edgar Allan Poe
- **HPL** – H. P. Lovecraft
- **MWS** – Mary Wollstonecraft Shelley

The dataset was particularly suitable for an LSTM because its primary input consists of written language. Text is sequential data because the position and order of words

influence meaning. A model attempting to recognise writing style therefore needs to consider more than simply whether individual words appear in a passage.

An LSTM is a type of recurrent neural network designed to retain useful information across a sequence. In this business problem, this means that the model can process a sequence of words and attempt to learn recurring language patterns associated with different authors.

The text variable was consequently selected as the predictive input, while author was selected as the target variable. The id variable was excluded from model training because it is only an identifier and provides no meaningful information about writing style.

Link to dataset: <https://www.kaggle.com/competitions/spooky-author-identification/data>

3. Analysis Plan and Data Preparation

The analysis was planned around five main activities: exploratory data analysis, feature selection, model training, model evaluation and reporting.

Exploratory data analysis was used first to establish whether the dataset was sufficiently clean and appropriate for modelling. Apache Spark was used to examine the number of observations and variables, inspect the schema, identify missing values and duplicate records, investigate the distribution of the three authors, and analyse the length of the text passages. Understanding these characteristics was necessary before model development because data-quality problems or significant class imbalance could affect the reliability of the model.

For feature selection, text was selected as the predictive variable because the purpose of the model is to identify an author from the characteristics of their writing. The author variable was selected as the target, while id was excluded because an arbitrary identifier does not describe writing style. Traditional feature-selection methods such as p-values were not considered suitable for the raw sequential text used in this model. Instead, feature preparation involved tokenising the text, selecting a vocabulary and converting passages into sequences of equal length.

The modelling stage was planned in two phases. A baseline LSTM would first establish an initial performance benchmark. If the results indicated weaknesses such as underfitting or poor classification across authors, the model would then be retrained using different architecture and hyperparameter settings.

Performance was planned to be evaluated using **accuracy, precision, recall and F1-score**, together with a classification report and confusion matrix. Training and validation accuracy and loss were also monitored to determine whether the model was learning effectively and whether there was evidence of overfitting or underfitting.

Finally, the results would be interpreted from the client's perspective. The purpose was not only to report numerical performance but also to determine whether the resulting model would be sufficiently reliable for the proposed author-guessing application.

```
[3]: file_path = "train.csv"

df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .option("multiline", "true")
    .option("escape", "'")
    .csv(file_path)
)

print("Dataset loaded successfully.")
```

Dataset loaded successfully.

1.4 Initial Dataset Inspection

The first five observations are displayed to confirm that the dataset has been loaded correctly and to understand its structure. The dataset contains an identifier, a text passage and an author label. The text represents the information that will be analysed by the model, while the author represents the class that the model will attempt to predict.

```
[4]: df.show(5, truncate=100)
```

```
+-----+-----+-----+
|      id|                                     text|author|
+-----+-----+-----+
[id26305]|This process, however, afforded me no means of ascertaining the dimensions of my dungeon; as I mi...|EAP|
[id17569]|It never once occurred to me that the fumbling might be a mere mistake.|HPL|
[id11008]|In his left hand was a gold snuff box, from which, as he capered down the hill, cutting all manne...|EAP|
[id27763]|How lovely is spring As we looked from Windsor Terrace on the sixteen fertile counties spread ben...|MWS|
[id12958]|Finding nothing else, not even gold, the Superintendent abandoned his attempts; but a perplexed l...|HPL|
+-----+-----+-----+
```

only showing top 5 rows

1.5 Dataset Dimensions

The number of rows and columns is calculated to determine the overall size of the dataset. This step is particularly important because the portfolio requires the selected dataset to contain a minimum of 10,000 records. The output confirms that the dataset contains sufficient observations for the required analysis and model development.

```
[5]: row_count = df.count()
      column_count = len(df.columns)

      print("Number of rows:", row_count)
      print("Number of columns:", column_count)
      print("Columns:", df.columns)
```

```
Number of rows: 19579
Number of columns: 3
Columns: ['id', 'text', 'author']
```

1.6 Dataset Schema

The Spark schema is examined to identify the variables contained in the dataset and their associated data types. Reviewing the schema confirms that Spark has interpreted the dataset correctly before further analysis is conducted. The `text` and `author` variables are particularly important because they will later become the predictive feature and target variable respectively.

```
[6]: df.printSchema()

root
 |-- id: string (nullable = true)
 |-- text: string (nullable = true)
 |-- author: string (nullable = true)
```

1.7 Missing-Value Analysis

Missing and blank values are investigated because observations without a valid text passage or author label cannot be used to train the supervised classification model. Missing values could reduce data quality and potentially cause problems during text preprocessing and model training. Any records containing missing or blank values in the required variables will therefore be removed during the cleaning stage.

The missing-value analysis identified 0 missing or blank values. This information was considered during the subsequent data-cleaning process.

```
[7]: missing_values = df.select([
      count(
        when(
          col(c).isNull() |
          (trim(col(c).cast("string")) == ""),
          c
        )
      ).alias(c)
      for c in df.columns
    ])

missing_values.show()

+---+-----+
| id|text|author|
+---+-----+
| 0| 0| 0|
+---+-----+
```

1.8 Duplicate Analysis

The dataset is examined for duplicate observations. Duplicate passages may cause certain examples to appear more frequently than intended and could influence the patterns learned by the model. Removing duplicate observations helps ensure that the analysis is based on unique records.

The analysis identified **0 duplicate observations** in the dataset.

```
[8]: total_records = df.count()
      unique_records = df.dropDuplicates().count()

      print("Total records:", total_records)
      print("Unique records:", unique_records)
      print("Duplicates:", total_records - unique_records)

Total records: 19579
Unique records: 19579
Duplicates: 0
```

1.9 Data Cleaning

The dataset is cleaned before further analysis and model development. Observations containing missing text, missing author labels, blank text, blank author labels and duplicate records are removed. These cleaning steps ensure that every remaining passage contains valid input text and a corresponding target class. The cleaned dataset will be used for the remainder of the exploratory analysis and model development.

The dataset contained **19579 observations** before cleaning and **19579 observations** after cleaning. This leaves a valid dataset that can be used for further analysis.

```
[9]: clean_df = (
      df
      .dropna(subset=["text", "author"])
      .filter(trim(col("text")) != "")
      .filter(trim(col("author")) != "")
      .dropDuplicates()
    )

      print("Records before cleaning:", df.count())
      print("Records after cleaning:", clean_df.count())

Records before cleaning: 19579
Records after cleaning: 19579
```

4. Exploratory Data Analysis

EDA

2.1 Author distribution

The number of passages belonging to each author is calculated to examine the distribution of the target variable. Understanding class distribution is important because severe class imbalance can influence classification performance. A model trained on an imbalanced dataset may favour the class containing the largest number of observations. The author counts will therefore be examined before model development.

```
[10]: author_counts = (
      | clean_df
      | .groupBy("author")
      | .count()
      | .orderBy(desc("count"))
      )

author_counts.show()

+-----+
|author|count|
+-----+
| EAP | 7900 |
| MWS | 6044 |
| HPL | 5635 |
+-----+
```

Exploratory data analysis was conducted to determine the quality of the dataset and identify characteristics that could influence model performance. The dataset was first examined for missing and duplicate observations. No missing or blank values were identified in the required variables, and all 19,579 observations were unique. Therefore, no records needed to be removed during the cleaning process, and the complete dataset could be retained for subsequent analysis.

The distribution of the target variable was then examined. EAP had the highest representation with 7,900 passages, followed by MWS with 6,044 passages and HPL with 5,635 passages. Although all three authors had a substantial number of observations, the distribution demonstrates a moderate class imbalance. EAP represents approximately 40% of the dataset and is therefore the majority class.

This difference was particularly important to identify before modelling. If a classifier struggled to learn meaningful differences between the authors, predicting the majority class repeatedly could produce an apparently reasonable overall accuracy despite poor performance for the other authors. This consideration later became important when interpreting the confusion matrix.

The author distribution was:

Author	Passages	Approximate share
Edgar Allan Poe (EAP)	7,900	40.35%

Mary Shelley (MWS)	6,044	30.87%
H. P. Lovecraft (HPL)	5,635	28.78%

2.2 Visualisation of Author Distribution

A bar chart is created to provide a visual comparison of the number of passages available for each author. The visualisation makes it easier to identify whether one author is substantially overrepresented or underrepresented. This information is important when interpreting the final model's accuracy, precision, recall and F1-score.

Interpretation:

The graph shows that **EAP** has the largest number of passages, while **HPL** has the smallest number. Overall, the classes appear **reasonably balanced**. This distribution will be considered when interpreting the model's classification performance.

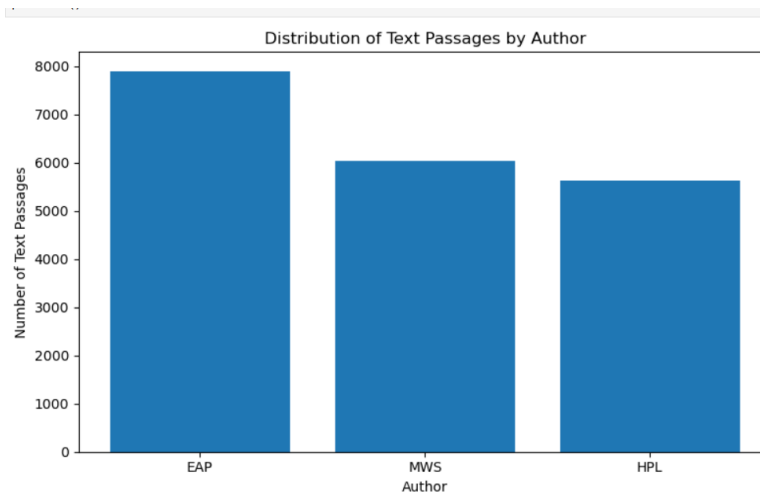
```
[11]: author_rows = author_counts.collect()

authors = [row["author"] for row in author_rows]
counts = [row["count"] for row in author_rows]

plt.figure(figsize=(8, 5))
plt.bar(authors, counts)

plt.xlabel("Author")
plt.ylabel("Number of Text Passages")
plt.title("Distribution of Text Passages by Author")

plt.tight_layout()
plt.show()
```



The classes are moderately uneven, but not severely imbalanced. EAP is the largest class. A classifier that always predicted EAP would achieve approximately 40.35% accuracy, which provides an important benchmark for interpreting the model results.

Passages contained an average of 26.73 words and 149.06 characters. The median passage contained 23 words, while passage length ranged from 2 to 861 words.

Average length also varied by author:

Author	Average Words	Average Characters
HPL	27.80	155.84
MWS	27.42	151.66
EAP	25.44	142.23

2.6 Average Passage Length by Author

The average word count and character count are calculated separately for each author. This analysis determines whether the authors differ in the average length of their available passages. Differences in passage length may represent one aspect of writing style, although the LSTM will primarily learn from patterns within the actual word sequences.

Interpretation:

The results indicate that **HPL** has the longest passages on average, while **EAP** has the shortest passages on average. This demonstrates that some differences in passage structure exist between the authors and may contribute to the patterns available to the classification model.

```
[14]: length_by_author = (
    eda_df
    .groupBy("author")
    .agg(
        avg("word_count").alias("average_words"),
        avg("character_count").alias("average_characters")
    )
    .orderBy(desc("average_words"))
)

length_by_author.show()

+-----+-----+-----+
|author|average_words|average_characters|
+-----+-----+-----+
|HPL|27.799645075421473|155.84347826086957|
|MWS|27.417273328921244|151.65982792852415|
|EAP|25.44240506329114|142.22594936708862|
+-----+-----+-----+
```

HPL passages were longest on average, while EAP passages were shortest. These differences suggest that passage structure may contain some author-related information. The wide range of passage lengths also justified padding the text sequences before training.

Overall, the EDA supported the modelling decisions. It showed that the data was sufficiently large and complete, that the classes were moderately uneven, and that the raw passages needed to be converted into a common numerical sequence format before they could be supplied to an LSTM.

2.3 Passage-Length Analysis

Two additional variables are created for exploratory purposes: `character_count` and `word_count`. Character count measures the number of characters contained in each passage, while word count provides an approximate measure of the number of words. Examining passage length helps identify differences in the size of the text observations and provides useful information when selecting the maximum sequence length for the LSTM.

```
[12]: eda_df = (
      clean_df
      .withColumn("character_count", length(col("text")))
      .withColumn("word_count", size(split(trim(col("text")), r"\s+")))
    )

eda_df.select(
  "author",
  "character_count",
  "word_count"
).show(10)
```

author	character_count	word_count
EAP	109	21
HPL	75	15
HPL	88	16
EAP	118	19
EAP	61	12
EAP	43	8
HPL	211	36
EAP	365	60
EAP	70	16
EAP	183	35

only showing top 10 rows

The EDA was conducted before the model was trained so that the results could inform the modelling process. The initial Spark display confirmed that the records contained genuine text and meaningful author labels. The schema check confirmed that the expected variables had been loaded. The missing-value analysis showed that no records required removal for missing text or labels, while the duplicate analysis showed that the observations were unique.

The author-count analysis was important because a classifier can appear accurate if it mainly predicts the largest class. EAP represented 40.35% of the data, compared with 30.87% for MWS and 28.78% for HPL. The classes were therefore not perfectly balanced, but the difference was not large enough to explain all model errors. This finding led to the decision to use a stratified split and to interpret the final model with class-level metrics.

The length analysis produced two additional EDA variables: `character_count` and `word_count`. The overall standard deviation was 19.05 words, showing meaningful variation around the mean of 26.73 words. The 25th percentile was 15 words, the median was 23 words and the 75th percentile was 34 words. Although most passages were short, the maximum of 861 words showed that a small number of passages were much longer than typical observations.

The author-level length results also showed a small structural difference. HPL passages averaged 27.80 words, MWS passages averaged 27.42 words and EAP

passages averaged 25.44 words. HPL passages were therefore approximately 2.36 words longer than EAP passages on average. This difference may provide a weak signal, but it is not sufficient evidence that length alone distinguishes the authors. The model was therefore trained on the original text rather than on length as a shortcut feature.

The EDA findings also identify a limitation in the choice of a 100-token maximum sequence length. It is long enough to cover most short passages, but it truncates any passage exceeding 100 tokens. Future analysis should measure how many passages are truncated and compare alternative sequence lengths. This would show whether important authorial information is being removed during preprocessing.

2.4 Passage-Length Summary Statistics

Descriptive statistics are calculated for the character count and word count variables. The minimum, maximum, mean and percentile values provide an overview of the typical size and variability of the text passages. The results also help determine whether very short or very long passages are present and support the decision to standardise sequence length before training the neural network.

```
[13]: eda_df.select(
      "character_count",
      "word_count"
    ).summary().show()
```

	character_count	word_count
count	19579	19579
mean	149.05740844782676	26.730476530977068
stddev	106.80018861273624	19.048353080110644
min	21	2
25%	81	15
50%	128	23
75%	191	34
max	4663	861

5. Model Training

Baseline model

The baseline model contained a 64-dimensional embedding layer, a 64-unit LSTM, 30%dropout, a 64-unit dense layer and a three-unit softmax output. It used the Adam optimiser with a learning rate of 0.001 and was trained for 10 epochs with a batch size of 64.

The embedding layer converted word indexes into trainable numerical vectors. The LSTM processed the word sequence, dropout reduced over-reliance on individual connections, and the softmax layer produced probabilities for the three authors.

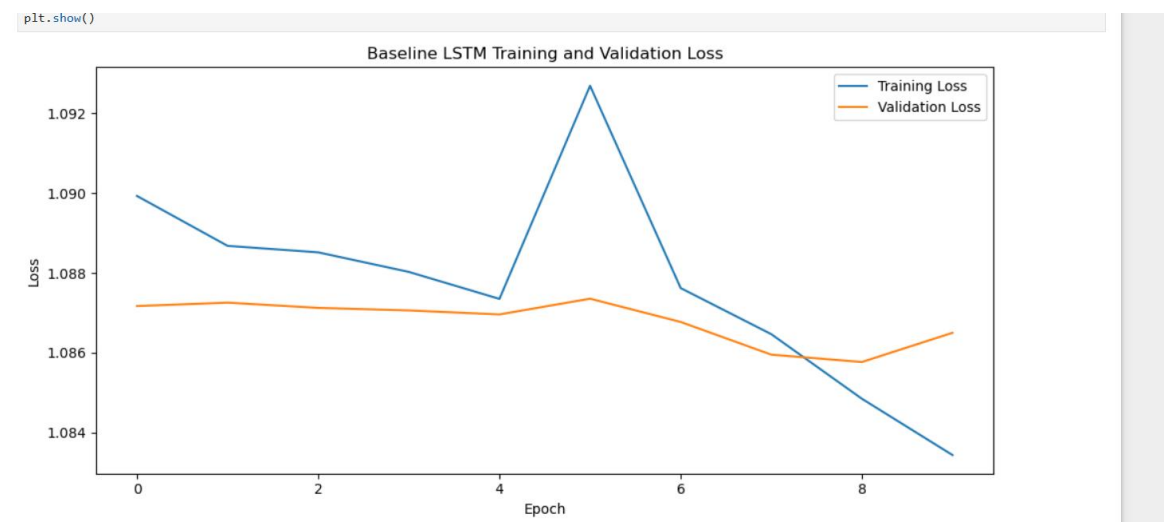
Sparse categorical cross-entropy was used because the target labels were integer encoded.

Training and validation accuracy remained close to 40%, and loss remained close to 1.09. This indicates underfitting because the model did not learn sufficiently discriminative patterns.



3.10 Baseline Training and Validation Loss

Training and validation loss are plotted across the training epochs. Loss measures the magnitude of the model's classification error. Ideally, both training and validation loss should decrease as the model learns. If training loss continues decreasing while validation loss begins increasing, the model may be learning the training data too specifically and therefore overfitting.



The improved model used a 128-dimensional embedding, a bidirectional 96-unit LSTM, a second 64-unit LSTM, increased dropout and a learning rate of 0.0005. Early stopping monitored validation loss. Training stopped after five epochs because

validation loss did not improve consistently. Both models used the same training and test data, making the comparison fair.

The changes were intended to learn richer word representations, use context in both directions, reduce overfitting and make training more gradual. However, the similar training behaviour indicates that the main problem was not simply a lack of model complexity.

6. Evaluation and Interpretation

The models were evaluated on the unseen test set using accuracy, precision, recall, weighted F1-score and a confusion matrix. Accuracy measures overall correctness, precision measures the reliability of predictions, recall measures how many examples from each class are found, and F1-score balances precision and recall

Metric	Baseline	Improved
Accuracy	0.4037	0.4035
Weighted precision	0.3392	0.1628
Weighted recall	0.4037	0.4035
Weighted F1-score	0.2339	0.2320

```
[28]: baseline_precision = precision_score(
    y_test,
    baseline_pred,
    average="weighted",
    zero_division=0
)

baseline_recall = recall_score(
    y_test,
    baseline_pred,
    average="weighted",
    zero_division=0
)

baseline_f1 = f1_score(
    y_test,
    baseline_pred,
    average="weighted",
    zero_division=0
)

print("Accuracy:", baseline_accuracy)
print("Precision:", baseline_precision)
print("Recall:", baseline_recall)
print("F1-score:", baseline_f1)

Accuracy: 0.40372830629348755
Precision: 0.33919135899871794
Recall: 0.40372829417773237
F1-score: 0.2338682014928391
```

```
[36]: improved_precision = precision_score(
    y_test,
    improved_pred,
    averages="weighted",
    zero_division=0
)

improved_recall = recall_score(
    y_test,
    improved_pred,
    averages="weighted",
    zero_division=0
)

improved_f1 = f1_score(
    y_test,
    improved_pred,
    averages="weighted",
    zero_division=0
)

print("Improved Accuracy:", improved_accuracy)
print("Improved Precision:", improved_precision)
print("Improved Recall:", improved_recall)
print("Improved F1-score:", improved_f1)

Improved Accuracy: 0.4034729301929474
Improved Precision: 0.1627904065038954
Improved Recall: 0.4034729315628192
Improved F1-score: 0.2319822532275307
```

The baseline achieved **40.37% accuracy**, only slightly above the 40.35% majority-class benchmark. Its F1-score of **0.2339** indicates poor balance across the authors.

The improved model did not improve performance. Accuracy fell slightly, precision fell substantially and F1-score also decreased. The baseline is therefore retained as the comparative final model, although neither model is ready for deployment.

The per-author baseline results were also uneven: EAP achieved precision of 0.40, recall of 1.00 and F1-score of 0.57; HPL achieved zero precision, recall and F1-score; and MWS achieved precision of 0.57, recall of 0.00 and F1-score of 0.01. This shows why the weighted results must be interpreted with the confusion matrix.

Actual/Predicted	EAP	HPL	MWS
EAP	1,577	0	3
HPL	1,127	0	0
MWS	1,205	0	4

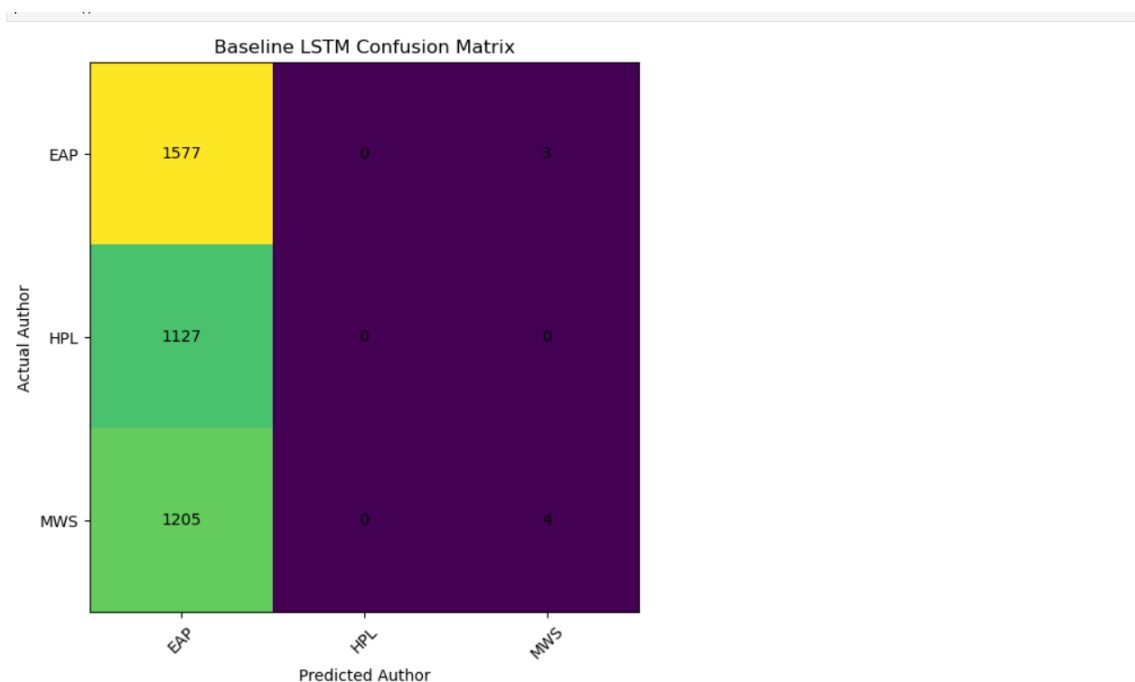
The classification report shows that **EAP** achieved the strongest classification performance, while **HPL** was more difficult for the model to classify correctly. This indicates that some author writing styles may contain more distinctive patterns than others.

```
[29]: print(
      classification_report(
          y_test,
          baseline_pred,
          target_names=label_encoder.classes_,
          zero_division=0
      )
)
```

	precision	recall	f1-score	support
EAP	0.40	1.00	0.57	1580
HPL	0.00	0.00	0.00	1127
MWS	0.57	0.00	0.01	1209
accuracy			0.40	3916
macro avg	0.32	0.33	0.19	3916
weighted avg	0.34	0.40	0.23	3916

The matrix shows that the model predicted almost every passage as EAP. It correctly classified 1,577 of 1,580 EAP passages, but none of the 1,127 HPL passages and only 4 of the 1,209 MWS passages. The model therefore collapsed toward the largest class rather than learning to distinguish all three authors.

This is a serious weakness for the proposed application. A reader submitting an HPL or MWS passage would almost always receive an incorrect EAP prediction. The model did not simply confuse HPL and MWS with one another; it failed to produce meaningful predictions for either class.



7. Limitations and Business Recommendation

The evaluation used one stratified train-test split. Repeated stratified cross-validation would provide a more reliable estimate of performance. Future experiments should also report macro F1-score and balanced accuracy because weighted averages can hide poor performance in smaller classes.

The preprocessing was limited to word-level tokenisation, a 15,000-word vocabulary and a maximum sequence length of 100 tokens. Future work should test character-level features, punctuation, different sequence lengths and alternative vocabulary sizes. The dataset also contains only three historical authors, so the results cannot be generalised to modern authors, non-fiction or other genres.

The current results do not justify deploying the LSTM as a customer-facing “Guess the Author” classifier. The model has not demonstrated balanced recognition of the three authors and performs close to a system that always predicts EAP. Before development continues, Large Book Press should compare the LSTM with TF-IDF logistic regression and a linear support-vector classifier. Class weighting, macro-averaged metrics and additional authors should also be investigated.

For an internal demonstration, the current model can still show the complete workflow from Spark-based EDA to neural-network evaluation. If presented to stakeholders, it should be labelled as an experimental prototype and clearly limited to EAP, HPL and MWS. A public version should only be considered after recall and F1-score improve substantially for all three authors.

8. Conclusion

The analysis demonstrated a complete Python workflow for author classification. The dataset was justified, inspected and cleaned using Spark. EDA established the class distribution and passage-length characteristics. The text was tokenised and padded before baseline and improved LSTM models were trained and evaluated on unseen data. The dataset was suitable for the investigation, but the models were not sufficiently effective. The baseline achieved 40.37% accuracy and a weighted F1-score of 0.2339, while the improved model achieved 40.35% accuracy and a weighted F1-score of 0.2320. The baseline is retained as the final model for this report only because it performed marginally better than the improved model.

The most important finding is that both models largely predicted EAP. The confusion matrix showed zero correctly classified HPL passages and only four correctly classified MWS passages. The LSTM should therefore be treated as a research benchmark rather than a finished product.

The report follows the analytical workflow from the business objective and dataset justification through planning, EDA, model training, evaluation and recommendation. Tables and figures provide evidence for the class distribution, training behaviour and confusion pattern. The conclusions are based on the Python outputs rather than on training performance alone.

The recommendation is written for the client: retain the baseline as the comparative final model, but do not deploy either model until balanced performance improves. This gives Large Book Press a clear explanation of what was achieved, what the main weakness is and what should be done next.

REFERENCES

Kaggle (n.d.) Spooky Author Identification: Dataset description and data fields. Available at: <https://www.kaggle.com/competitions/spooky-author-identification/data> [Accessed: 25 August 2026].

Scikit-learn developers (n.d.) Model evaluation: Classification metrics and confusion matrices. Available at: https://scikit-learn.org/stable/modules/model_evaluation.html [Accessed: 28 August 2026].

TensorFlow (2024) tf.keras.layers.LSTM API documentation. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM [Accessed: 25 August 2026].

Apache Spark (n.d.) PySpark documentation. Available at: <https://spark.apache.org/docs/latest/api/python/> [Accessed: 26 August 2026].

Link to Github: <https://github.com/ST10388321/PDAN8412w-Part-1>